

Taller de Programación I

TA045 - Curso: Deymonnaz

Grupo 5: Zoidberg

Alumnos:

- Juan Pablo Dominguez Lucia - 111401
- Julen Leonel Gaumard - 111379
- Juan Jose Torga - 111339
- Felipe Gazcón - 110933

Corrector:

- Tomas Cichero

Docente a cargo:

- Fernando Oreiro

Contacto:

- Sebastián Balbarani

[Enlace a repositorio](#)

Introducción	3
Arquitectura del proyecto	3
Client	3
Server	4
Simulations	5
Crates de Soporte	6
Decisiones de diseño clave	7
Base de datos	7
Cache	8
Concurrencia	8
Servidor HTTP	8
Autenticación	9
Gestion de docentes	10
Configuraciones	10
Features de la aplicación Web	11
Login	11
Lado profesor	11
Lado Alumno	12
Herramientas adicionales implementadas	13
Implementaciones interesantes	14
La marea	14
Desviación del barco por olas	14
Mapa del fondo	15
Complicaciones durante el desarrollo	16
Interpolación	16
Cache	17
Base de datos y concurrencia	17
Librerías Utilizadas	18
Conclusiones	19
Anexo: Evolución del proyecto	20

Introducción

Se nos presentó la siguiente situación, hay un docente Fernando Oreiro, de la materia Levantamientos Hidrográficos, para la carrera de Agrimensura, el cual tiene la siguiente dificultad, al momento de enseñar batimetría es complicado explicar el efecto de los distintos instrumentos solo de forma didáctica, lo ideal es realizar una experiencia real donde los alumnos pueden experimentar con las distintas sondas, debido al reducido presupuesto de la universidad, esta opción es imposible, ya que, en reducidas veces se logra hacer solamente 1 excursión en donde el docente muestra cómo se realiza la medición de forma correcta.

La idea del proyecto es permitir al alumno, mediante una página web, experimentar con las distintas sondas y variables y simular mediciones. Esto permite, que cuando hagan la única experiencia en persona, comparar y validar las conclusiones u observaciones que realizaron con la plataforma.

La plataforma consiste de un servidor y una página web donde el docente sube modelos reales del fondo marino en formato GeoTIFF y los asigna a cada grupo de alumnos. Los alumnos, sin ver el fondo real, configuran los parámetros de una ecosonda virtual y un recorrido de relevamiento sobre esa zona. El sistema simula cómo se vería esa medición aplicando ruidos según lo seleccionado y finalmente interpola los puntos muestreados para reconstruir el fondo.

El objetivo pedagógico es que el alumno entienda, en la práctica, las decisiones que se toman antes de salir a relevar.

Arquitectura del proyecto

El proyecto está organizado en un workspace de rust dividido en 3 crates

Client

Es la interfaz web del sistema, escrita en Yew y compilada a WebAssembly, estilizada con Tailwind. Implementa las dos vistas principales de la plataforma: la pantalla del profesor y la pantalla del alumno.

- **main.rs**: punto de entrada de la SPA en WASM.
- **router.rs** / **protected_route.rs**: ruteo entre páginas y protección de rutas según autenticación.
- **parser.rs**: validación y parseo de los inputs de parámetros que ingresa el usuario.
- **components/**: piezas de UI reutilizables en toda la app (modales, botones, títulos, inputs de formulario).
- **structs/**: tipos de datos del cliente.

- `state.rs`: el estado global de la simulación en curso.
- `limits.rs`: los rangos válidos de cada parámetro, recibidos del server.
- `project.rs` / `student.rs`: representación local del proyecto y del alumno.
- **`services/`**: comunicación con el server.
 - `api_client.rs` / `api_utils.rs`: el cliente HTTP base (fetch) y sus utilidades.
 - `requests.rs`: las funciones concretas que llaman a cada endpoint del server (login, traer proyecto del alumno, correr simulación, etc.).
 - `utils.rs`: utilidades generales del módulo.
- **`pages/`**: las pantallas principales, una carpeta por rol.
 - `login/`: pantalla de inicio de sesión, con formularios separados para alumno y profesor.
 - `admin/`: panel del profesor, con secciones para gestión de proyectos y de alumnos.
 - `student/`: pantalla de simulación del alumno, con sus componentes (parámetros de ecosonda, transporte, recorrido, visor de imagen, fondo de mapa, leyenda de colores).
 - `not_found.rs`: página de error 404.

Server

Implementa el backend HTTP con `tiny_http`, maneja la persistencia de `SQLite` y expone los endpoints que consume el cliente. Incluye autenticación con tokens, hashing de contraseñas con `bcrypt` y un sistema de cache para evitar recalculación de simulaciones repetidas

- **`main.rs`**: punto de entrada del binario, levanta el servidor HTTP.
- **`structs/`**: tipos internos del server.
 - `settings.rs`: configuración leída de `config.toml` (rutas, claves, límites).
 - `request.rs`: tipos para el manejo de cada request entrante y su logging.
 - `filecache.rs`: caché en memoria de simulaciones/matrices ya calculadas.
 - `student_project_response.rs`: el struct de respuesta que arma `/student_project`, con las coordenadas del geotiff.
- **`db/`**: persistencia en SQLite.
 - `engine.rs`: el motor de conexión a la base.

- `migrations.rs`: el esquema de tablas (proyectos, alumnos, profesores).
 - `encrypt.rs`: hashing de contraseñas.
 - `queries/`: las consultas SQL crudas, una por entidad (`auth.rs`, `professor.rs`, `projects.rs`, `student.rs`).
 - `queries_interface/`: la capa intermedia que envuelve esas queries con el lock del mutex de la base antes de ejecutarlas.
- **`requests/`**: la capa HTTP.
 - `handler.rs`: el router principal, asigna cada request a su endpoint según método y ruta.
 - `http_helper.rs`: utilidades para parsear el body de las requests.
 - `endpoints/`: la lógica de cada grupo de rutas (`auth.rs`, `projects.rs`, `simulation.rs`, `students.rs`, `limits.rs`, `webpage.rs`, `generic.rs` con las respuestas comunes de error/éxito).
 - **`threads/`**: manejo de concurrencia, cada request se atiende en un hilo nuevo.
 - **`utils/`**: funciones de apoyo transversales.
 - `config_loader.rs`: lectura del archivo de configuración.
 - `helpers.rs` / `helpers_endpoints.rs`: utilidades generales y de autenticación compartidas entre endpoints.

Simulations

Carga y procesa los GeoTIFF con la librería GDAL, genera el recorrido de relevamiento, calcula los puntos de medición según el modo de ecosonda, aplica las perturbaciones físicas pertinentes e interpola el resultado generando imágenes de salida.

- **`lib.rs`**: El punto de entrada del crate. Expone las funciones públicas que usa el `server` (correr una simulación completa, generar el recorrido, calcular las coordenadas del geotiff, etc.) y orquesta el llamado a todo lo demás.
- **`structs/`** :
 - el struct `DepthMatrix`, la representación interna del geotiff cargado (la matriz de profundidades + metadata: ancho, alto, `no_data`, `geotransform`, proyección).
 - `student_measuring_parameters.rs`: los parámetros que selecciona el estudiante
 - `interpolation_type.rs` : Enum que distingue el tipo de interpolación a usar.
 - `measurement_type.rs` : Enum que distinguen los sistemas de ecosonda y como guardar sus mediciones.

- **processing/**: el grueso de la lógica, dividido en cuatro sub-áreas:
 - **geotiff.rs**: carga el GeoTIFF con GDAL, arma la **DepthMatrix**, y calcula las coordenadas lat/lon de las esquinas (lo que armamos hace un rato).
- **routing/**: genera el recorrido del barco:
 - **route.rs**: arma la trayectoria según azimuth y separación entre líneas.
 - **gnss.rs**: le agrega el ruido de posicionamiento al recorrido. Simulando desvíos por olas al barco.
- **measuring/**: toma las mediciones a lo largo del recorrido:
 - **points.rs**: elige qué puntos del recorrido se miden, según la distancia configurada.
 - **beam.rs**: calcula qué celdas de la matriz cubre cada pulso del sonar, para monohaz (un punto) y multihaz (una franja)
 - **disturbances.rs**: aplica todas las perturbaciones físicas a cada medición:
 - Potencia y ganancia
 - Sensor inercial
 - Marea
 - Error de velocidad del sonido
 - Umbral de detección.
- **interpolation/**: reconstruye el fondo a partir de los puntos medidos:
 - **interpolation_handler.rs**: orquesta el proceso de interpolación dependiendo de qué sistema de ecosonda se haya elegido para medir (multihaz, monohaz).
 - **interpolation_method/**: las distintas implementaciones: **tin_gdal.rs** es la que se usa hoy (delega a **gdal_grid**), y **old_interpolations/** guarda las versiones viejas (IDW, Kriging, TIN manual), código que ya no se usa pero quedó como referencia.
 - **helpers.rs** → funciones de apoyo compartidas.
- **images/**: genera las imágenes PNG de salida:
 - **simulation_image.rs**: la imagen coloreada de la simulación interpolada (con la escala rojo, azul de profundidad).
 - **path_image.rs**: la imagen del recorrido y visualización de la zona cubierta por el tipo de medición seleccionada.
 - **helpers.rs**: el mapeo de profundidad a color y demás utilidades compartidas de dibujo.

Crates de Soporte

Además, se incluyen dos crates más que complementan al sistema: **common**, con los tipos de datos compartidos entre los demás módulos, y **cli**, una herramienta de administración por línea de comandos.

- **common**: crate de tipos compartidos entre **server**, **client** y **simulations** (parámetros de ecosonda, transporte, recorrido, modos de medición). evita duplicar las mismas estructuras en cada lado.
- **cli**: herramienta de línea de comandos para administración del sistema. Se autentica contra el **server** vía HTTP y ofrece un menú interactivo para tareas administrativas, sin necesidad de pasar por la interfaz web.

Decisiones de diseño clave

Base de datos

Para la persistencia se eligió SQLite, una base liviana que no necesita un servidor aparte y queda como un único archivo.

El esquema es simple: una tabla de **professors**, una de **projects** (con su path al geotiff asociado y los parámetros configurados por el profesor), una de **students** (vinculados a un proyecto), y una de **auth_tokens** para manejar las sesiones activas.

El acceso está organizado en capas: las consultas SQL en sí viven separadas de la lógica que las invoca, y esa capa intermedia es la encargada de tomar el lock del **Mutex** que protege la conexión antes de ejecutar cualquier query. Para las contraseñas se usa **bcrypt**, evitando guardar nada en texto plano.

Se decidió generar una estructura la cual es la encargada de instanciar la conexión y ejecutar las **Query**. Dentro de la estructura, se establecieron los métodos de la misma con acceso solo al crate **pub(crate)**. Luego se generaron métodos, organizados según la entidad o tabla que administran, los cuales reciben parámetros, arman la **Query** en **SQL** y la ejecutan. Estos métodos funcionan de intermediarios entre la base de datos y los endpoints, introduciendo una capa de abstracción, la cual desacopla la lógica de negocio de los detalles de persistencia.

```
Professors(id, username, password_hash, price_boat, price_launch, price_ship,
price_echosounder_monohaz, price_echosounder_multihaz, price_sensor_sound_profiler,
price_sensor_mareograph, price_sensor_inertial)
```

```
Projects(id, name, description, filename, attempts_limit, weather, seabed_hardness, budget,
geotiff_min_depth, geotiff_max_depth, professor_id)
```

```
Students(id, name, code, attempts, professor_id, project_id)
```

```
Auth_tokens(id, professor_id, student_id, token, created_at, expires_at)
```

Cache

Uno de los procesos costosos que debe ejecutar el servidor, es cargar los geoTIFF y transformarlos en nuestra estructura, para luego procesarlo. Como los grupos van a estar simulando una y otra vez (cada vez que el alumno prueba una configuración distinta), tiene sentido no repetir esa carga en cada request.

Por eso se decidió implementar una caché en memoria con un límite de tamaño configurable, el cual guarda la matriz ya cargada de los proyectos usados más recientemente. Descartando el que lleve más tiempo sin usarse, si se encuentra lleno.

Además de la matriz, también cachea el último recorrido calculado para un proyecto, así que si el alumno vuelve a pedir el mismo recorrido con los mismos parámetros, tampoco hace falta recalcularlo.

Al estar compartido entre todos los hilos del servidor, este caché vive protegido por el mismo mecanismo de lock `Arc<Mutex<T>>`. El cual garantiza su correcto funcionamiento, aun bajo accesos concurrentes.

Concurrencia

El punto donde más concurrencia aparece es en el `server`.

El modelo es simple: por cada conexión que llega, se levanta un **hilo nuevo** (`thread::spawn`) que se encarga de atenderla. No hay límite de hilos, si entran cien requests al mismo tiempo, se crean cien hilos. Esto se hizo, porque se decidió no usar `async` en el crate que maneja las solicitudes HTTP.

Hay dos cosas puntuales que se comparten entre usuarios y necesitan de locks para protegerlas:

- **La base de datos:** la conexión a SQLite vive detrás de un `Arc<Mutex<DBEngine>>`. Cada hilo, antes de leer o escribir en la base, tiene que tomar el lock, así nunca hay dos accesos pisándose al mismo tiempo.
- **El caché de simulaciones:** las matrices de profundidad ya cargadas (para no leer el GeoTIFF de disco en cada request) también están protegidas por su propio `Arc<Mutex<FileCache>>`, por la misma razón.

El uso de `Arc` en los dos casos es lo que permite que todos los hilos tengan acceso al mismo dato sin copiarlo, y el `Mutex` adentro es lo que evita que dos hilos lo modifiquen a la vez.

Servidor HTTP

Para el backend se descartó usar un framework web como Axum, y se optó por construir el servidor sobre `tiny_http`, una librería que solo se encarga de aceptar conexiones y

exponer el request/response. Todo lo demás como el ruteo, el parseo del body, la autenticación, el manejo de errores se realizó con implementaciones propias.

El ruteo, es un único `match` que combina el método HTTP y la ruta pedida (por ejemplo, `(Method::Post, "/run_simulation")`) y delega cada caso al endpoint correspondiente. Esto tiene la contra de tener que escribir a mano cosas que un framework daría gratis, como la extracción de parámetros, pero a cambio el comportamiento del servidor es fácil de seguir.

```
12 pub fn handle_request(mut request: Request, cache: Arc<Mutex<FileCache>>, db: Arc<Mutex<DBEngine>>, settings: Arc<Settings>) -> RequestLog {
13
14     let result: (Response<Box<dyn Read + Send>>, _) = if let Some(api_path: &str) = request.url().strip_prefix(API_V1) {
15
16         match (request.method(), api_path) {
17
18             (Method::Options, _) => {
19                 (Response::empty(status_code: 200).boxed(), 200, None)
20             }
21
22             _ => {
23                 match (request.method(), api_path) {
24                     (Method::Post, "/create_path") =>
25                         simulation::create_path(&mut request, cache, db, settings),
26
27                     (Method::Post, "/run_simulation") =>
28                         simulation::run_simulation(&mut request, cache, db, settings),
29
30                     (Method::Post, "/coverage_image") =>
31                         simulation::create_coverage_image(&mut request, cache, db, settings),
32
33                     (Method::Get, "/limits") =>
34                         limits::get_limits(settings),
35
36                     (Method::Get, "/projects") =>
37                         projects::get_projects(&mut request, db),
38
39                     (Method::Post, "/projects") =>
40                         projects::create(&mut request, db, settings),
41
42                     (Method::Delete, url: &str) if url.starts_with("/projects/") =>
43                         projects::delete_project(&mut request, db, settings),
44
45                     (Method::Put, url: &str) if url.starts_with("/projects/") =>
46                         projects::update_a_project(&mut request, db),
47
48                     (Method::Get, "/student_project") =>
49                         projects::get_student_project(&mut request, db, settings),
50
51                     (Method::Get, "/students") =>
52                         students::get_all_students(&mut request, db),
53
54                     (Method::Post, "/students") =>
55                         students::create_new_student(&mut request, db),
56
57                     (Method::Delete, url: &str) if url.starts_with("/students/") =>
```

Autenticación

Desde un principio se planteó que distintos grupos/alumnos iban a tener que simular distintos proyectos, por este motivo se consideró necesario algún sistema de autenticación, a su vez, el docente también requiere alguna clase de control, ya que es el encargado de crear proyectos y administrar alumnos.

Considerando opciones, en conversaciones con el ayudante se propuso usar el Auth de Google, ya usado en otras áreas de la universidad. Se decidió descartar esta idea, debido a que nos exponemos a que un cambio en alguna de sus APIs rompa el funcionamiento del simulador y no podemos asegurar que, en un futuro, haya otros alumnos o desarrolladores para hacer mantenimiento si es que es requerido.

Por este motivo autenticar de la siguiente manera:

- Alumnos/Grupo: Código de 6 dígitos generado automáticamente
- Docentes: Usuario y contraseña

Ambos generan un token, el cual es almacenado en una cookie por el navegador.

Gestión de docentes

Los docentes crean, modifican y gestionan a los grupos/alumnos. Pero, ¿Quién gestiona a los docentes? Esa fue una pregunta que nos surgió al comenzar el diseño del esquema de la base de datos.

La idea de crear una página web exclusiva, con su propio modelo de autenticación, nos parecía sumamente excesiva. Por eso decidimos simplemente crear el CLI, descrito en anterioridad en el apartado de Crates de soporte. La autenticación del mismo existe y es sólo necesaria cuando se lo usa fuera de la computadora que ejecuta el servidor, requiriendo una contraseña establecida en el archivo de configuración del proyecto.

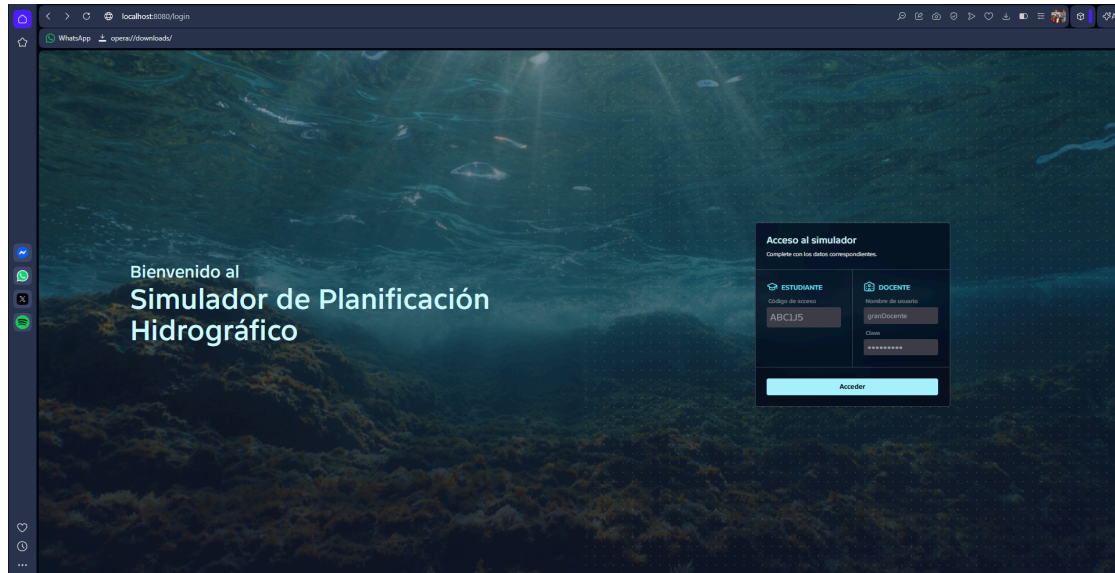
Configuraciones

Como cualquier servidor web, hay una serie de variables que no deseamos que estén hardcodeadas, como por ejemplo el puerto o algunas key de servicios externos. Además, este proyecto en particular, posee muchas variables extra de límites para las distintas sondas y sistemas de medición.

Por este motivo, se implementó un parser que procesa un archivo .toml, cargando las distintas variables e instanciando una estructura con todas las variables.

Features de la aplicación Web

Login



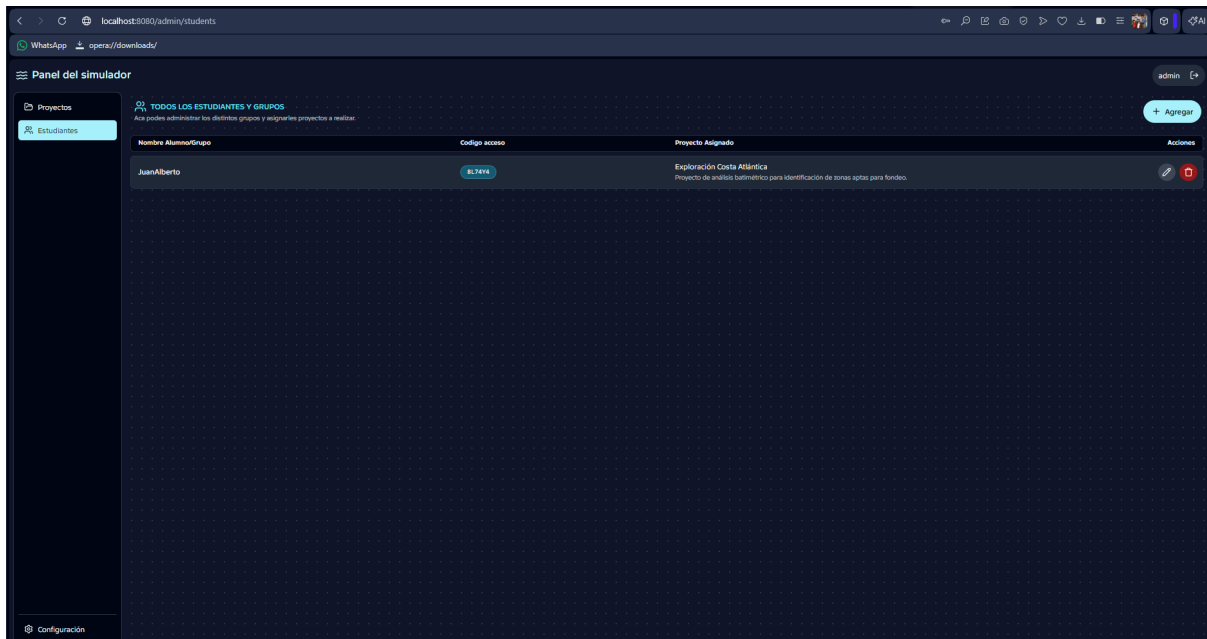
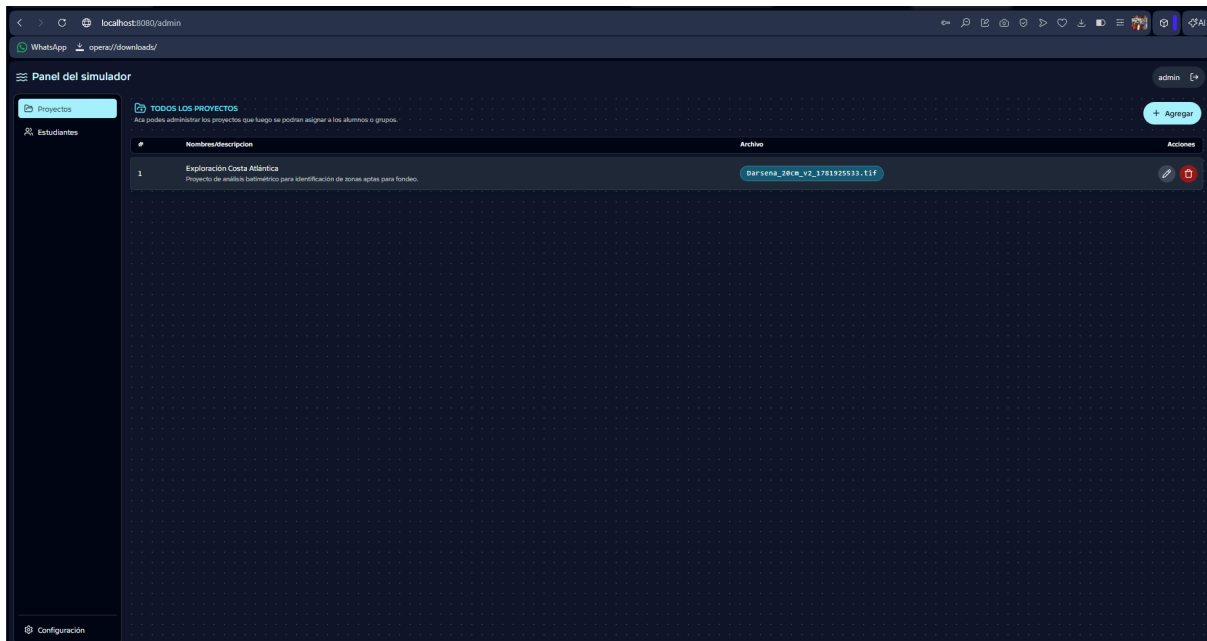
La aplicación posee dos interfaces distintas dependiendo de si el que ingresa es el profesor o un alumno.

El Inicio de sesión está separado y es genérico para los dos. El mismo se encargará de redirigir al alumno o al docente a la página que corresponda una vez autenticado.

Lado profesor

Si se ingresa como profesor, se puede:

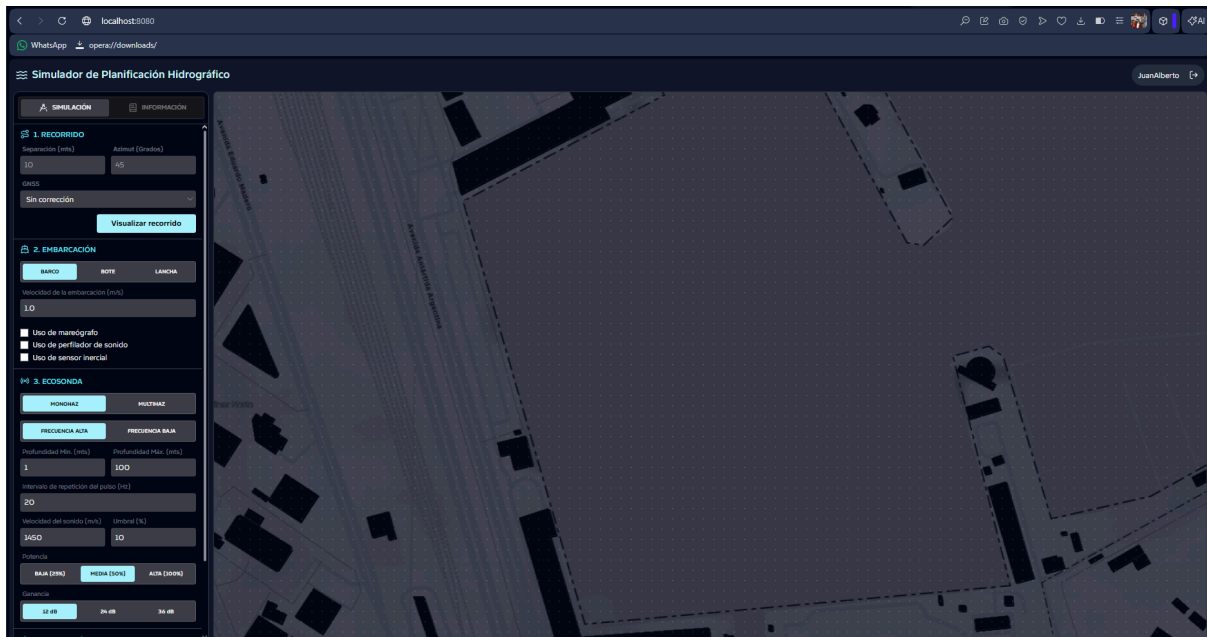
- Crear proyectos nuevos, subiendo el GeoTIFF de la zona a relevar.
- Configurar los parámetros de cada proyecto: límite de intentos, condición climática, dureza del fondo, presupuesto asignado, etc.
- Editar y borrar proyectos existentes.
- Ver el listado completo de proyectos creados.
- Gestionar alumnos: verlos en una tabla, asignarles el proyecto que les corresponde, editarlos o borrarlos.
- Ver el código que hay que repartirle a cada grupo para que puedan entrar a simular.



Lado Alumno

Si se ingresa como alumno, se puede:

- Configurar el recorrido: separación entre líneas, azimut, y tipo de corrección GNSS.
- Elegir la embarcación y su velocidad, y activar o desactivar mareógrafo, perfilador de sonido y sensor inercial.
- Configurar la ecosonda: modo (monohaz/multihaz), frecuencia, rango de profundidad, intervalo de repetición del pulso, potencia y ganancia.
- Visualizar el recorrido antes de correr la simulación.
- Correr la simulación y ver el resultado como imagen de profundidades, con su escala de colores.
- Ver la cobertura de la medición sobre la zona.



Herramientas adicionales implementadas

Aparte de la página web, el sistema tiene una herramienta de línea de comandos pensada para tareas de administración que no tiene sentido exponer en una interfaz gráfica. Pensado principalmente para la gestión de docentes.

```

juanpi@Juampi:~/zoidberg$ cargo run -p cli -- localhost:3000 Magicpass2
Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.41s
Running 'target/debug/cli 'localhost:3000' Magicpass2'

Iniciando sesion como admin...

SIMULADOR HIDROGRAFICO CLI

Bienvenido al CLI del simulador. Este permite añadir docentes y restaurar contraseñas.

Crear un nuevo usuario.
└─ create <username> <pass>

Cambiar contraseña de usuario.
└─ newpass <username> <newpass>

Cerrar todas las sesiones.
└─ closeall

>

```

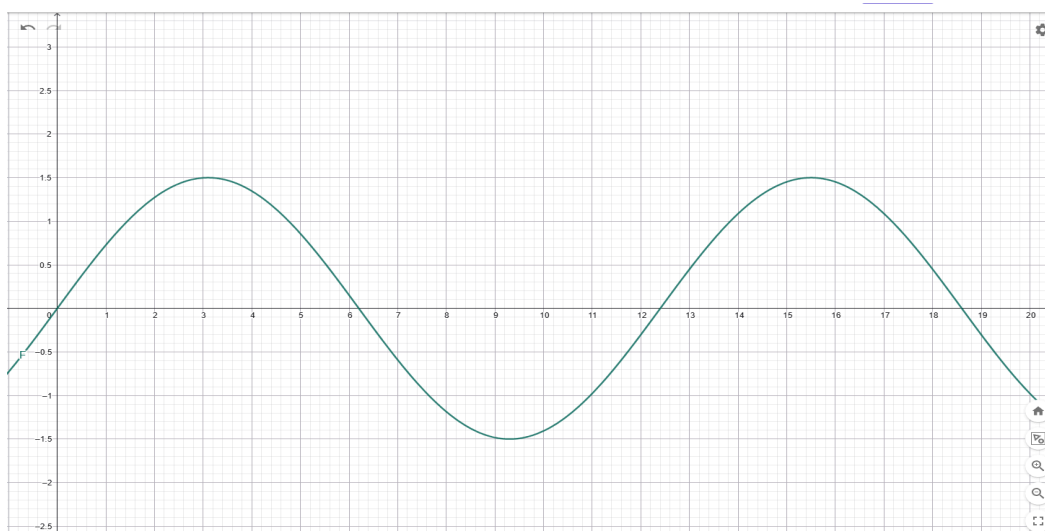
Para entrar, te pide la dirección del servidor y la contraseña de admin, si se logra autenticar, se presentará un menú con tres comandos: **create** para crear un usuario de docente nuevo (pasándole usuario y contraseña), **newpass** para resetear la contraseña de un docente que la perdió, y **closeall** para cerrar todas las sesiones activas de una sola vez.

Implementaciones interesantes

La marea

Para simular la marea, usamos una onda sinusoidal que sube y baja cada 12.4 horas, que es el ciclo real de la marea. Se calcula una sola vez según **cuánto tiempo real tarda todo el recorrido**, que sale de dividir la distancia total por la velocidad del barco. Después, cada punto medido toma el valor de marea que le toca según en qué momento del recorrido fue tomado.

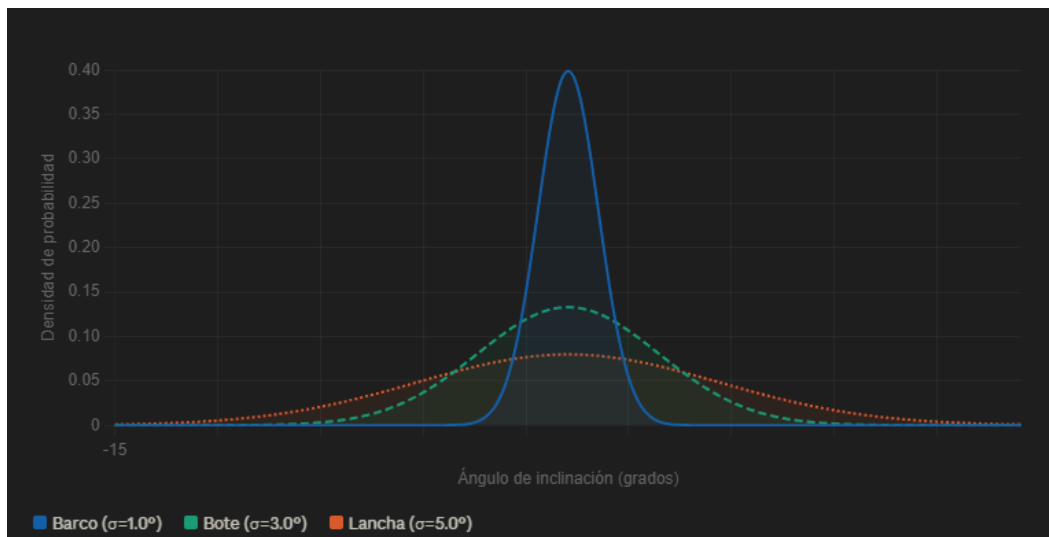
Lo único que realmente hace que la marea se note es que el recorrido tarde más, o sea, ir más lento con el barco. Y si el alumno tiene prendido el mareógrafo, este efecto directamente no se aplica, porque se asume que ese instrumento ya está corrigiendo la marea en el momento.



Eje Y es la amplitud, eje X es el tiempo en horas.

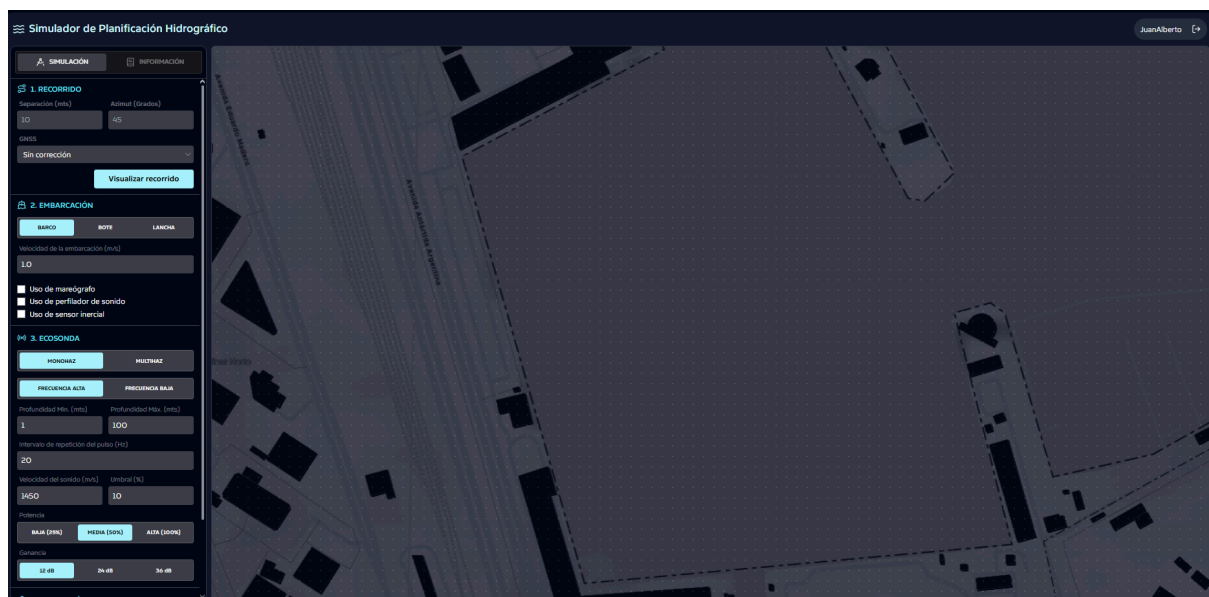
Desviación del barco por olas

Este sensor simula que el barco no siempre apunta el sonar perfectamente derecho hacia abajo, porque se mueve, se inclina un poco con las olas. Esa inclinación se genera con un sorteo al azar que sigue una campana (distribución normal): la mayoría de las veces la inclinación es chica, pero a veces puede ser más grande, y nunca tiene una dirección fija. Qué tan grande puede llegar a ser esa inclinación depende de dos cosas: el tipo de embarcación (un barco grande se mueve menos que una lancha) y la velocidad (cuanto más rápido se va, más se sacude). Si se activa el sensor inercial, esta desviación no ocurre



Mapa del fondo

Para que el alumno vea en qué parte del mundo está la zona que le tocó medir, le agregamos un mapa de fondo (usando MapTiler) que se centra solo y ajusta el zoom automáticamente según el tamaño real de la zona. Para lograr esto, el servidor calcula las coordenadas (latitud y longitud) de las cuatro esquinas del archivo del fondo marino, y utiliza esa información, para establecer la posición y zoom que debe tener el mapa.



Complicaciones durante el desarrollo

Interpolación

La interpolación es la parte del sistema que toma los puntos medidos por la ecosonda simulada y reconstruye toda la superficie del fondo a partir de ellos

Uno de los problemas que más tiempo nos llevó resolver fue encontrar un método de interpolación que funcionase.

Probamos varios métodos antes de llegar al que quedó en la versión final. Empezamos con una implementación propia de **IDW**, que fue lo primero que nos permitió tener algo andando, pero el resultado era lento a medida que crecía la cantidad de puntos. Intentamos armar una segunda versión de IDW apoyada en una estructura kd-tree para acelerar la búsqueda de puntos cercanos, pero al tener una gran cantidad de puntos muy cercanos entre sí (una medición por cada píxel del recorrido), el kd-tree tiraba panic.

Para solucionarlo, armamos una función que reducía la cantidad de puntos antes de pasarlos al kd-tree: dividimos la matriz en celdas de 75x75 píxeles, y de cada celda nos quedamos con un solo punto representativo en lugar de todos los medidos ahí. Pero esto traía dos problemas:

1. A mayor cantidad de puntos medidos, más había que reducir (más agresiva era la selección dentro de cada celda), resultando en una peor medición. En realidad debería ser al revés: más mediciones tendrían que dar más precisión, no menos.
2. El tamaño de la celda debía escalar según el tamaño del geotiff. 75x75 podía quedarse corto en geotiffs grandes (sin reducir lo suficiente) o ser demasiado grande en los de baja resolución (descartando casi toda la información).

Probamos con **Kriging**, un método más sofisticado estadísticamente, pero bastante más pesado de calcular. El resultado fueron varios minutos de espera solo para una interpolación inclusive con el kdtree.

Llegamos a implementar también una versión de **TIN** hecha a mano la cual mostraba mejoras en la visualización respecto a IDW y era mas rapida que Kriging, sin embargo seguía siendo muy lenta para lo que esperábamos.

Después de varias vueltas, y notar que nos estábamos quedando cortos de tiempo decidimos delegar la interpolación a `gdal_grid`, una herramienta de la misma librería GDAL que ya usábamos para leer los geotiffs. Funciona como un proceso externo: exportamos los puntos medidos a un archivo, lo invocamos como subprocesso, y leemos el resultado. Es bastante más rápida que cualquiera de nuestras versiones, y al ser una herramienta madura y probada, el resultado visual también salió mejor.

Cache

Conforme fuimos desarrollando y testeando la simulación, descubrimos que algunos pasos de la misma son demandantes, y dependiendo el hardware en el que se ejecuten, llegar a tardar hasta un minuto en realizar la simulación completa. Como la idea es que el alumno esté, en su clase, constantemente experimentando, sentimos que esa larga espera es en detrimento de la educación. Por ese motivo nos sentamos a debatir, para encontrar una forma de reducir los tiempos, la primera solución, reducir la complejidad de los algoritmos la descartamos rápidamente, ya que en tests observamos que las mediciones introducen demasiado error, lo cual no podemos permitir.

En este mismo proceso de análisis y testeo, observamos que algunos de los procesos que demoran bastante tiempo, son los de cargar el archivo y procesarlo o el de generar el recorrido, el cual, para un proyecto dado no cambia. Por ese motivo se decidió implementar una estructura compartida entre threads que hace de caché, el cual, según lo configura, almacena los últimos DepthMatrix y Path generados. De esta manera, cuando algún grupo o alumno intenta simular, el server primero comprueba si la información ya se encuentra disponible ahí, llegando a reducir a la mitad el tiempo de simulación, dependiendo de los parámetros elegidos.

Base de datos y concurrencia

Para la base de datos usamos el crate `sqlite`, el cual, según la documentación, maneja los accesos concurrentes, con flags en el mismo archivo, por lo cual levantábamos una conexión a la misma, en cada thread que recibía una request. A la vez, sabemos que, como es una herramienta académica, es muy probable que en un instante haya muchos accesos concurrentes, por ejemplo, cuando el profesor anuncia en clase que están subidos los proyectos y todos los alumnos entran a simular, por lo cual decidimos hacer pruebas para confirmar que el manejo que hacia el crate era óptimo.

Para las pruebas desarrollamos un pequeño script en Python, el cual hace varias requests en un tiempo reducido, de unas decenas de milisegundos. Ahí mismo descubrimos que para periodos tan cortos, el manejo de parte del crate no es el óptimo y puede bloquear el acceso a la base de datos, como podrá observar en la imagen:

```
Request 2
Status: 500
Tiempo: 0.005s
Body: No se pudo iniciar la db
-----
Request 1
Status: 404
Tiempo: 0.036s
Body: 404 no existe
-----
```

Como se observa, uno de las requests respondió con error 500, ya que no pudo abrir la conexión a la DB. La solución que encontramos fue implementar un control de concurrencia con `Arc<Mutex<T>>`, como se mencionó en anterioridad.

Librerías Utilizadas

common

- `serde` — serialización/deserialización de los tipos compartidos entre cliente y servidor.

simulations

- `gdal` — lectura y manipulación de archivos GeoTIFF.
- `image` — generación de las imágenes PNG de resultado.
- `spade` / `kiddo` — estructuras geométricas (triangulación, kd-tree) usadas en las primeras versiones de interpolación propia.
- `nalgebra` — álgebra lineal para cálculos geométricos.
- `rand` / `rand_distr` — generación de números aleatorios y distribuciones estadísticas (la normal del sensor inercial, el ruido GNSS, etc.).

server

- `tiny_http` — servidor HTTP minimalista, base de todo el backend.
- `sqlite` — persistencia en base de datos.
- `bcrypt` — hashing seguro de contraseñas.
- `rand` — generación de tokens de sesión.
- `hex` — codificación hexadecimal (tokens/hashes).
- `multipart` — parseo del formulario de subida de archivos (GeoTIFF).
- `base64` — codificación de datos binarios para transporte.
- `serde` / `serde_json` — serialización de requests/responses

client

- `yew` — framework de componentes para la SPA, compilado a WebAssembly.
- `yew-router` — ruteo entre páginas (login, alumno, profesor).
- `gloo-net` — cliente HTTP (fetch) para hablar con el servidor.
- `gloo-timers` — temporizadores en el navegador.
- `wasm-bindgen` / `js-sys` / `web-sys` — interoperabilidad entre Rust/WASM y las APIs del navegador (DOM, eventos, blobs).
- `lucide-yew` — íconos de la interfaz.

cli

- `request` — cliente HTTP para comunicarse con el servidor desde la terminal.

Conclusiones

En primer lugar, este proyecto nos requirió instruirnos en áreas que no son de común conocimiento para los Ingenieros Informáticos, eso nos requirió investigación y periódicas reuniones con los docentes para entender los distintos aspectos de la batimetría. Aunque no implicó un tiempo y un esfuerzo extra, consideramos que fue sumamente interesante y energizante que el dominio del proyecto no sea uno conocido.

A su vez, durante estos meses, fue de una dificultad particular avanzar el proyecto equilibrando lo esperado por la materia y lo deseado por el profesor, el cual podemos considerar nuestro cliente. Desde un principio tuvimos que negociar con Fernando los términos y alcances que el programa debería abarcar, entendiendo que el resultado debía ser orientado a la enseñanza y práctica. Por otro lado Tomas, nuestro profesor corrector, nos advertía que áreas necesitaban más trabajo, y recomendaba pasos para seguir y tecnologías a las que investigar.

El producto final es un programa de simulación de actividades batimétricas, aplicadas a entornos académicos donde los alumnos podrán experimentar y aplicar lo aprendido antes de salir a ejercer en el mundo real. Nos basamos en recursos y tecnologías con validación en el mundo real, empezando por Rust y herramientas de software como GDAL, WASM y otras herramientas de desarrollo como Bruno.

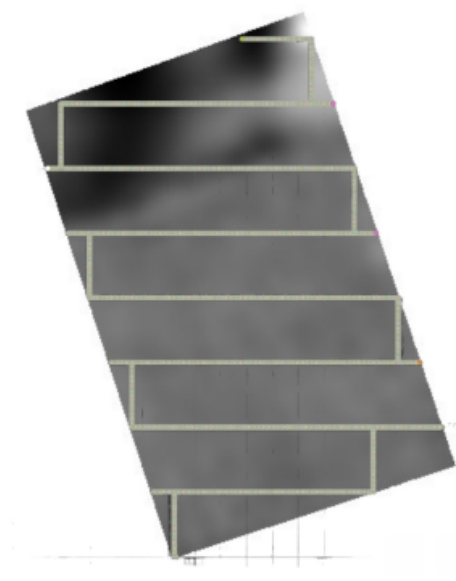
Creemos que logramos cumplir con las necesidades y expectativas del cliente y a su vez, pudimos aplicar los distintos conocimientos vistos durante la cursada. Además, aunque existan muchas características posibles a agregar, consideramos que pudimos crear un recurso de gran utilidad y valor para la carrera de Agrimensura y la universidad.

Anexo: Evolución del proyecto

El proyecto empieza en la base lógica del sistema: el geotiff y el procesamiento de los datos de este para ser incorporados. Se creó el archivo geotiff.rs, una de las bases del proyecto por la minúscula cantidad de cambios que ha tenido y la importancia que tiene al ser responsable de la formación del struct DepthMatrix, primordial para nuestro programa. Este archivo fue implementado haciendo uso de nuestro primer crate importante, el GDAL, librería estándar para la manipulación de archivos de información geoespacial.

```
1  #[derive(Clone)]
   1 implementation
2  pub struct DepthMatrix {
3      pub data: Vec<Vec<f64>>,
4      pub width: usize,
5      pub height: usize,
6      pub no_data: Option<f64>,
7      pub size_x: f64,
8      pub size_y: f64,
9      pub geo_transform: [f64; 6],
10     pub projection: String,
11 }
```

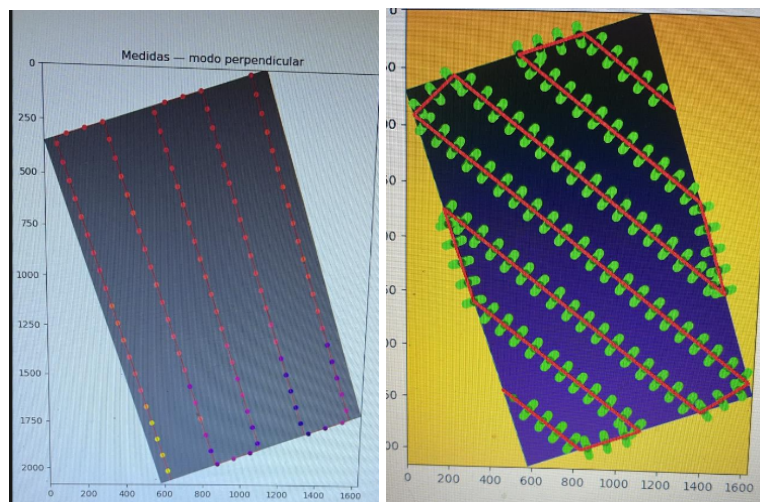
Seguido, se empezó a pensar uno de los aspectos clave de la simulación, la ruta de la embarcación. Se creó el archivo routing.rs, que usando el struct DepthMatrix mencionado previamente, y los parámetros de azimuth y distancia entre piernas generaba una ruta a través de toda la matriz. Este archivo, al igual que varios otros, fue modificado una cantidad considerable de veces. En instancias tempranas, tardó en desarrollarse un trayecto continuo y natural.



primer intento de recorrido, posteriormente corregido

Los dos aspectos más polémicos de la simulación fueron la toma de medidas y la interpolación. A esta altura, se empezó a desarrollar la primera. Antes de empezar con esto, hicimos unos scripts en python para testear más fácilmente como se iban aplicando nuestras medidas y cálculos en la matriz, haciendo más fácil la visualización de nuestro proceso. Dicho esto, empezamos haciendo un algoritmo que calcula, en base a la ruta previamente determinada y un parámetro de distancia entre medidas, todos los puntos de la matriz en los que luego se tomaría una medida.

Teniendo ya los puntos de medición, empezamos a desarrollar los métodos que simularán los tipos de sonda, Monohaz y Multihaz, y cómo estas determinan las áreas a medir y los puntos de la matriz que las conforman. Todas estas funciones relacionadas al cálculo del valor de cada punto tuvieron que ser rediseñadas un sinfín de veces, siendo las más polémicas de la simulación debido a su importancia en el realismo de esta.



Una captura de en una de las primeras pruebas monohaz y multihaz

Fue habiendo terminado ese segmento que empezamos a enfocarnos en lo ajeno a la simulación; el front-end y la arquitectura del servidor. Se hizo el primer gran refactor del proyecto, que incluye la creación de módulos para server, client, simulaciones y código en común, implicando también la incorporación de otros crates significativos para el desarrollo del sistema, incluyendo por ejemplo el WASM. Además, se empezaron a diseñar algunas de las firmas más relevantes del proyecto, las que conectaban el servidor y la librería de simulaciones, y las que comunicaban el servidor con el cliente. Esto fue hecho de manera anticipada para organizar y facilitar el trabajo a largo plazo, lo cual resultó ser muy necesario. Se asentaron las bases del manejo de errores y la comunicación entre procesos.

El segundo de los protagonistas polémicos de la simulación fueron los algoritmos de interpolación, empezando con una burda implementación de IDW pensada por nosotros.



Primer intento de interpolación

Esto fue un primer acercamiento, que utilizamos solo para probar el funcionamiento del resto del sistema sabiendo que este algoritmo era reemplazable por ende no era necesario tenerlo perfeccionado de inmediato, siendo que además es algo que se debe afilar constantemente.

A la vez se arrancó con lo visual del programa. Nuestros primeros avances front-end fueron los relacionados a la parametrización y ejecución de la simulación, siendo que en esto se centraba todo el programa. Se trabajó a la par la simulación y este segmento del front-end para agilizar la conexión entre ambas partes, dado que ambos lados necesitaban tener esto listo para poder seguir avanzando. Además organizamos el código en componentes reutilizables para hacerlo más limpio.

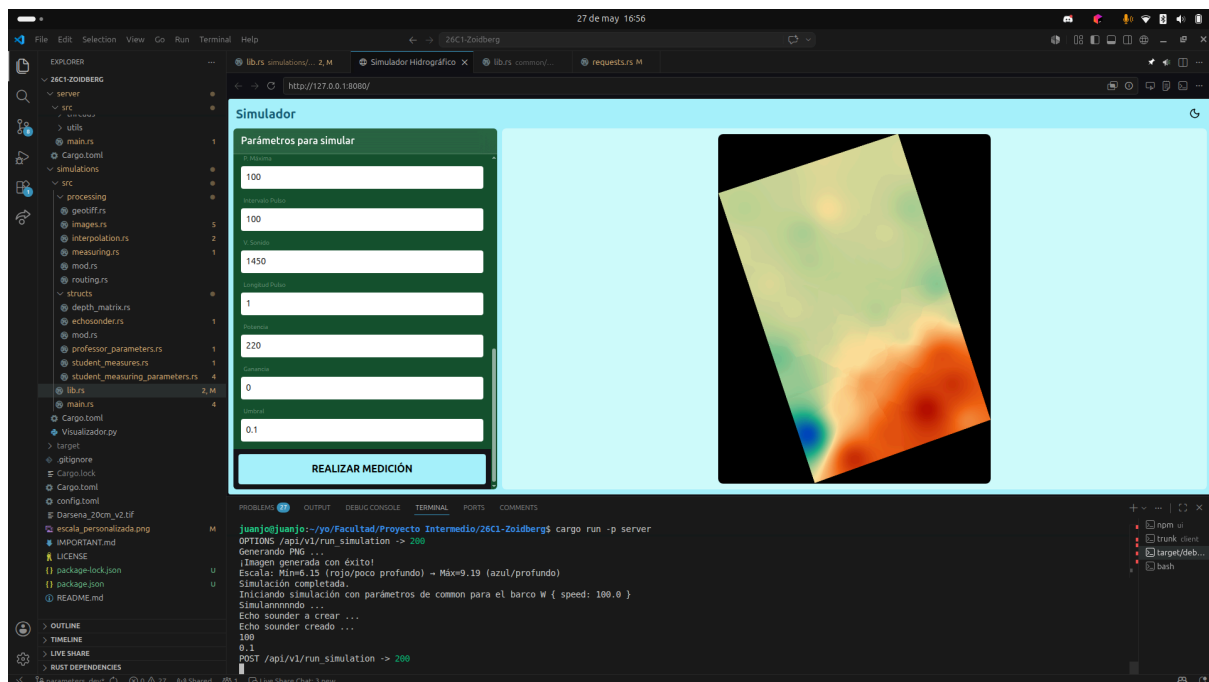
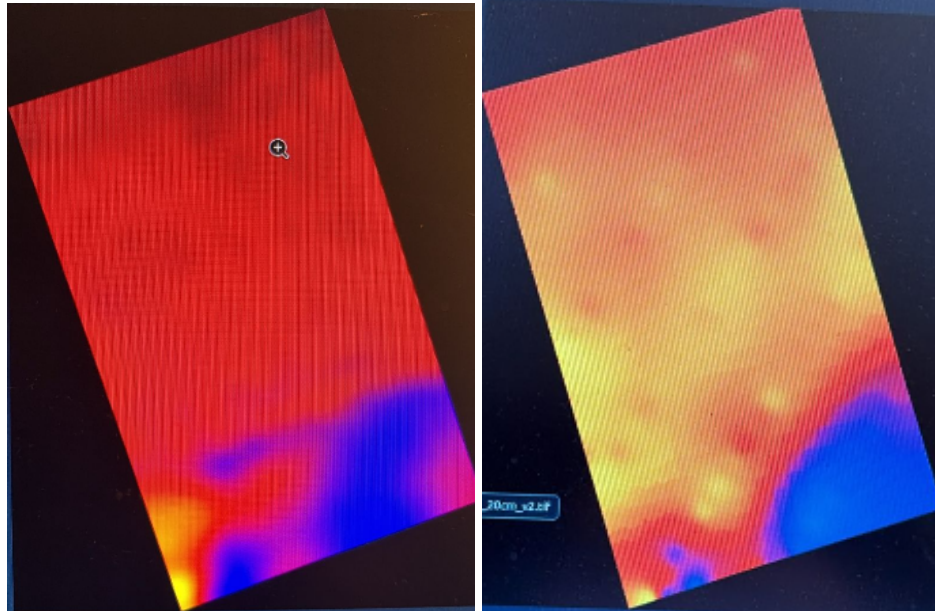


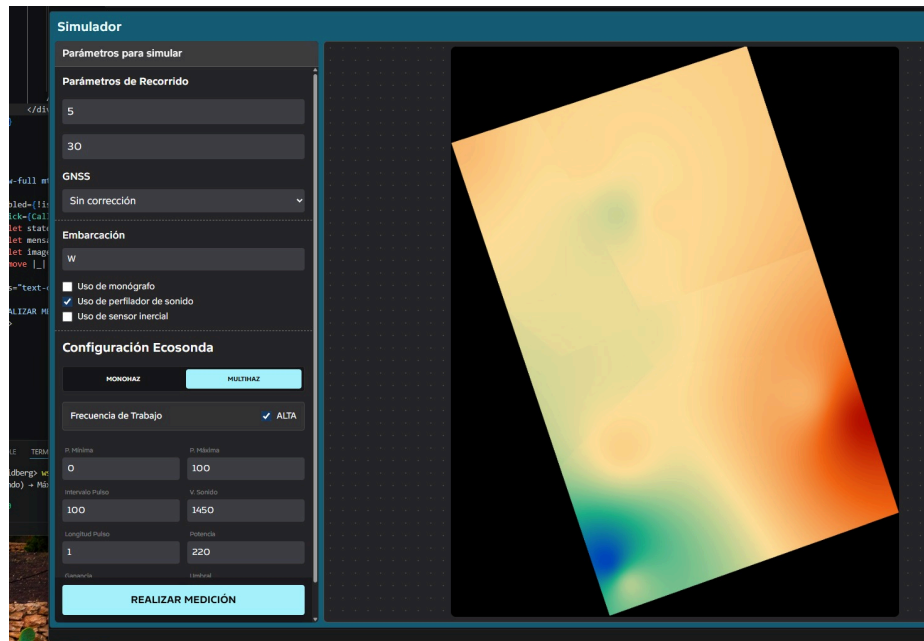
Imagen del front en una etapa temprana del desarrollo

En este tramo empezamos a avanzar por todos los frentes al mismo tiempo: front-end para el cliente, simulación y server. Para la simulación se crearon los algoritmos de Kriging, alternativa más demandante pero más precisa, y se generó otra versión del IDW basándose en estructuras de datos que optimizarán el tiempo de ejecución, el kdtree. El server empezó a desarrollar la base de datos y se pulieron algunas de las requests más importantes del sistema. Además, se elaboraron los algoritmos de creación de imágenes en base a las matrices dadas por el profesor, o producto de la interpolación.



Fotos enviadas por whatsapp sobre avances en la interpolación

Después de mucho avance incuestionado, tuvimos una tanda de reuniones con Fernando, profesor de la carrera y cliente de nuestro proyecto, en las que nos hizo replantear varios aspectos del código, especialmente funcionalidades que deberían estar disponibles para el usuario y aspectos del realismo de la simulación. Esto derivó en cambios a la toma de medidas, interpolación, coloreo de los mapas y configuraciones generales. Además, se añadió una carga considerable de trabajo al front y la logística del servidor, ya que hay que tener en cuenta que cada posible herramienta sugerida implicaba un cambio en todos los niveles del sistema.



Mejora en el front de la aplicación

Los cambios en general pueden ser resumidos a una reconfiguración de los parámetros y cómo estos se usaban. Este se volvió el foco principal del proyecto, priorizado primero reaprender que significaba cada parámetro y luego preparando el programa para que lo maneje bien internamente. Los resultados más importantes fueron los parámetros más sofisticados de simulación, la implementación del gnss, la reestructuración de nuevos parámetros en el servidor, una primera base de datos funcional, las bases de la caché y una nueva interfaz gráfica más acorde a lo requerido por el profesor.

Seguido de ese tramo más intenso, llegó la entrega intermedia del trabajo. Luego de dicha entrega, nos empezamos a enfocar en el servidor y el front, entendiendo que las simulaciones habían llegado al punto aceptable y podían ser mejoradas más adelante, por lo que solo necesitamos cambiarla en momentos donde necesitamos algo nuevo de esta. Los primeros pasos fueron planificar mejor los parseos y comunicaciones entre procesos, así como el manejo de los errores en las nuevas funcionalidades. Hubo mucho trabajo pesado en la implementación de la caché, y mejoras estéticas notorias en el front. Además, se creó una de las piezas más importantes del sistema, el CLI. Este significó un avance muy importante en el trabajo, y amerito una seguidilla de cambios en los endpoints y base de datos.

```
Running "target\debug\cli.exe"

SIMULADOR DE ECHOSONDA CLI

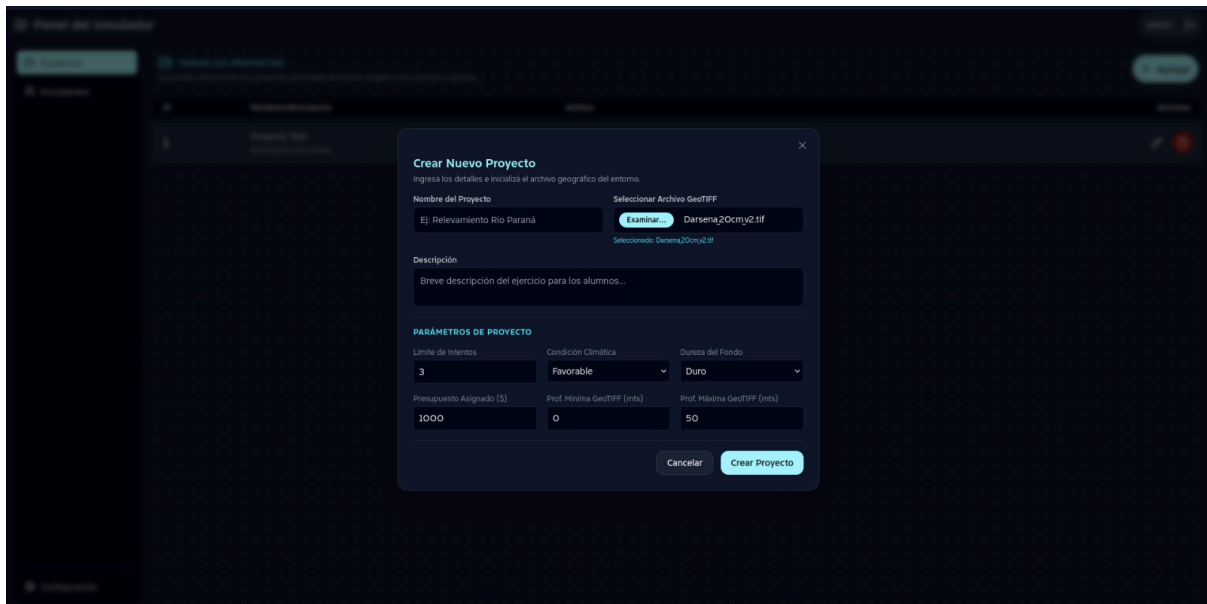
Desde aca podés agregar usuarios para docentes y restaurar tu contraseña

Crear un nuevo usuario.
└─ create <username> <pass>

Cambiar contraseña de usuario.
└─ newpass <username> <newpass>
```

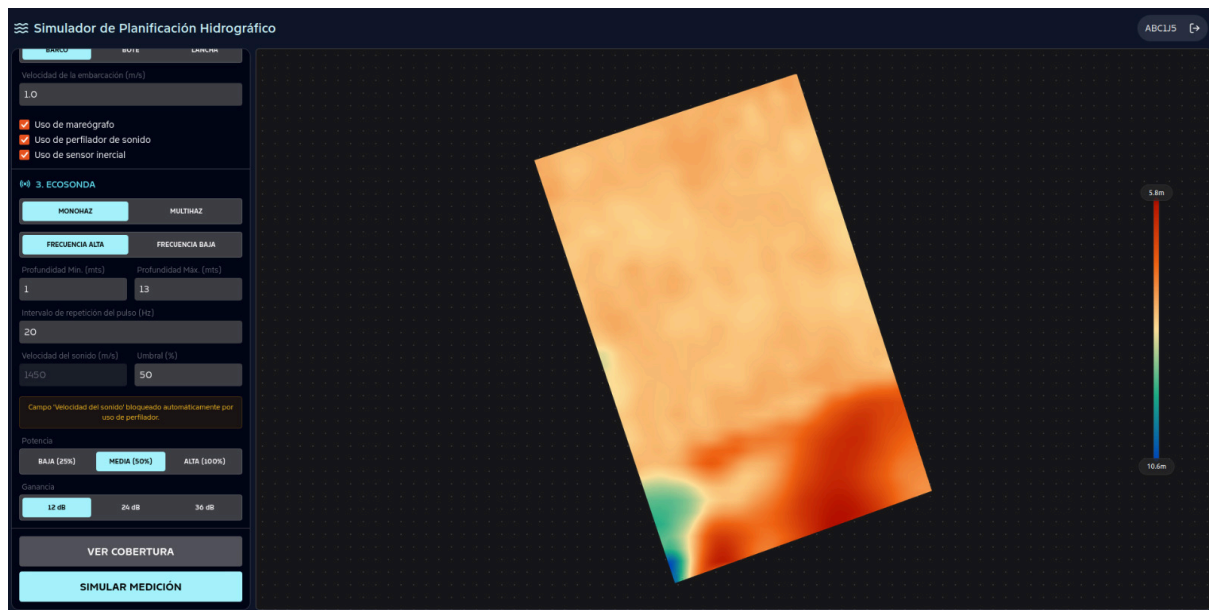
Herramienta CLI al ejecutarse

Cuando se incorporó casi completamente el CLI, volvimos a buscar mejoras en las otras áreas, sobre todo temas estéticos, por más que no siempre la mejora fuera hecha en el front sino más abajo. Entre otras cosas, hicimos nuevos algoritmos que generan imágenes de previsualización del recorrido, actualizamos la manera en la que el recorrido se calculaba, y por sobre todo se empezó a diseñar la faceta de login de la aplicación. Esto conlleva diseñar en el front, pero también crear todos los endpoints necesarios, hacer nuevas conexiones en la DB y crear una lógica de autenticación, que resulta en un esfuerzo enorme. Vale la pena mencionar que empezamos a utilizar Bruno, para testear las apis y la base de datos de manera local para agilizar el desarrollo.

A screenshot of a web application showing a modal window titled "Crear Nuevo Proyecto". The modal has a dark theme. It contains a form with the following fields: "Nombre del Proyecto" with the value "Ej: Relevamiento Rio Paraná", "Selección Archivo GeoTIFF" with a file selection button "Examinar..." and a selected file "Darsena20cmx2.tif", and a "Descripción" field with the placeholder text "Breve descripción del ejercicio para los alumnos...". Below these fields is a section titled "PARÁMETROS DE PROYECTO" which includes a table with three columns: "Límite de Intentos", "Condición Climática", and "Dureza del Fondo". The table has one row with values "3", "Favorable", and "Duro". Below the table are three input fields: "Presupuesto Asignado (\$)" with the value "1000", "Prof. Mínima GeoTIFF (mts)" with the value "0", and "Prof. Máxima GeoTIFF (mts)" with the value "50". At the bottom right of the modal are two buttons: "Cancelar" and "Crear Proyecto".

Modal de creación de proyecto para el profesor

Días más adelante, se siguen creando fragmentos de api, limpiando endpoints, generando más elementos visuales y reconsiderando la interpolación. Se llega al punto que decidimos enfocar nuestros esfuerzos en puntos concretos, por lo que empezó una nueva etapa un tanto más intensa. Uno de los resultados es que armamos una nueva interpolación, más efectiva y rápida que todas las anteriores basada en GDAL, librería que usamos previamente, y que acompañamos con una nueva forma de medir y calcular los puntos. También se realizó una gran parte del trabajo necesario para el logueo, la comunicación de parámetros, la creación y gestión de proyectos por parte del profesor, y la posibilidad de insertar nuevos tiffs al sistema, que es en cierto modo el objetivo del producto.



Una de las últimas versiones del front de simulaciones

Llegados a esta parte de la cronología, faltaban 10 días para la entrega del trabajo. Empezamos a aclarar los últimos pasos de la entrega del trabajo, y nos comprometimos con la limpieza y corrección del código. Se hicieron muchísimos avances más en el front, incluyendo animaciones nuevas, una nueva versión del login, la incorporación del logo y por sobre todo la introducción de las redirecciones funcionales en la página. Hubo más avances en la DB implementando gran parte del trabajo faltante en autenticaciones y gestión de los proyectos, alumnos y profesores. También hubo un gran trabajo en la finalización de la caché del sistema, que se amoldo al funcionamiento de la DB de manera que tratarasen de igual manera la información del sistema.



Logo de la aplicación y pantalla de login hecha por Julien

En la recta final se implementaron cosas como las imágenes de áreas cubiertas, autenticaciones en el CLI y endpoints de admin. Se crearon apis para la comunicación de datos menores del geotiff, pero de importancia para features estéticas nuevas. Se conectaron más entre front y back, e hicieron más retoques estéticos en la interfaz gráfica y las interpolaciones. El trabajo realizado estos últimos 10 días, es muchísimo mayor al comunicado y amerita revisar cada comité como un párrafo aparte.

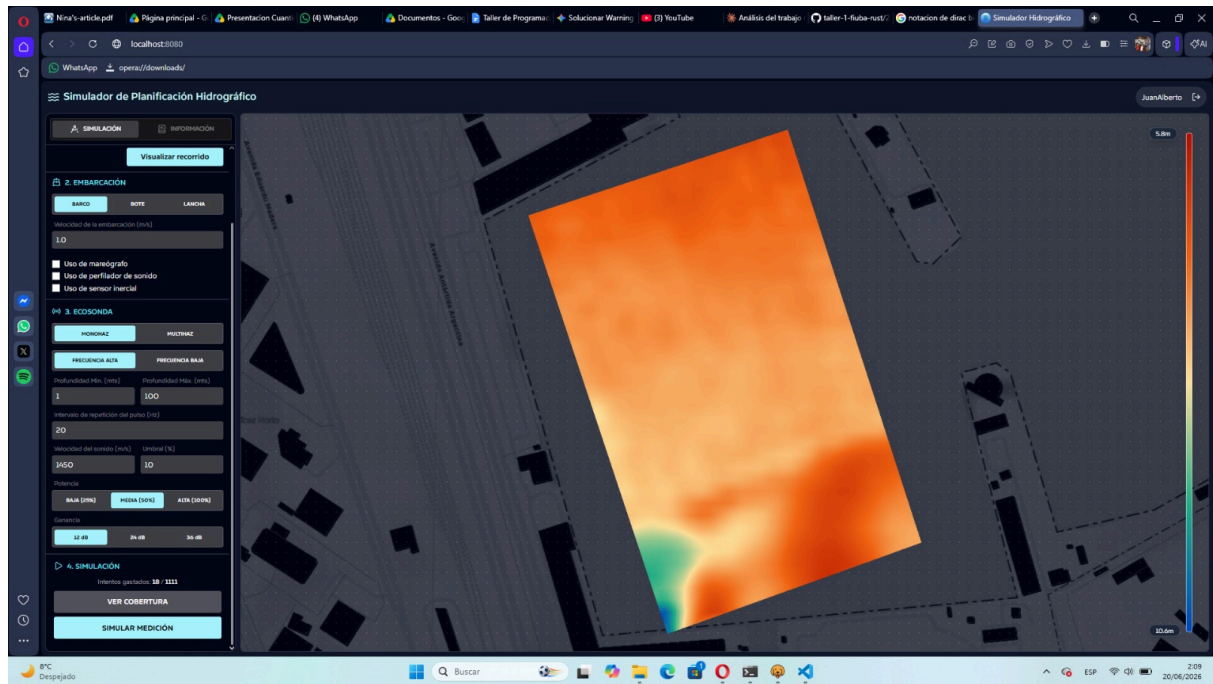


Imagen final del front de visualización